

MECHATRONICS AND IOT

ASSIGNMENT REPORT – 14

TITLE:

Design and develop a GPS-Based Vehicle Tracking and Fleet Management System. The system enables real-time vehicle tracking and efficient fleet monitoring for transportation management.

TEAM MEMBERS:

HARISANKARAN S

(24MER013)

MUTHUKUMAR M

(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

The GPS-Based Vehicle Tracking and Fleet Management System is designed to provide real-time information about the location and movement of vehicles. The system uses a GPS module, microcontroller, communication module, and monitoring platform to collect and transmit vehicle location data. The system helps fleet operators monitor vehicles, improve transportation management, track vehicle movement, and manage multiple vehicles efficiently.

PROBLEM STATEMENT

- Difficulty in tracking vehicle location in real time.
- Lack of continuous vehicle monitoring.
- Difficulty in managing multiple vehicles.
- Delayed information about vehicle movement.
- Inefficient fleet utilization.
- Difficulty in monitoring vehicle routes.
- Lack of centralized vehicle information.
- Increased possibility of unauthorized vehicle usage.
- Difficulty in maintaining efficient transportation operations.

PROPOSED SOLUTION

The proposed system uses a GPS module to determine the real-time geographical location of a vehicle. The GPS coordinates are processed by a microcontroller and transmitted to a remote monitoring platform through a communication module.

The system can provide:

- Real-time vehicle location.
- Latitude and longitude information.
- Vehicle movement monitoring.
- Route tracking.
- Fleet status monitoring.
- Vehicle identification.
- Location history.
- Efficient fleet management.
- Remote monitoring.

The fleet operator can monitor the location and status of vehicles from a centralized interface.

WORKING PRINCIPLE

The GPS module receives signals from GPS satellites and determines the vehicle's geographical position.

1. GPS module receives signals from satellites.
2. GPS module determines the vehicle's latitude and longitude.
3. Microcontroller reads the GPS coordinates.
4. The coordinates are processed by the controller.
5. Communication module transmits the location data.
6. The monitoring platform receives the data.
7. Vehicle location is displayed on the monitoring interface.
8. The fleet operator monitors vehicle movement.
9. The process continuously repeats to provide updated vehicle information.

SYSTEM ARCHITECTURE

GPS Satellite → GPS Module → Microcontroller → Communication Module → Cloud/Monitoring Platform → Fleet Operator

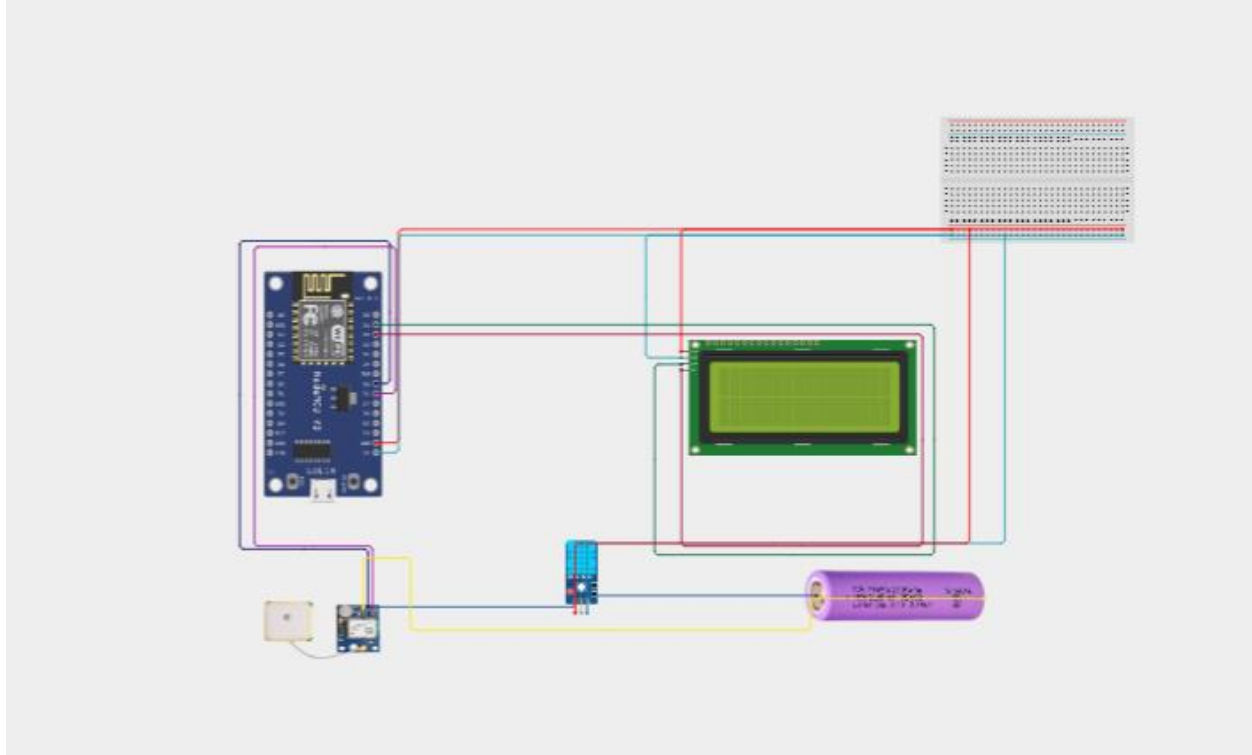
VEHICLE OPERATING CONDITIONS

Parameter	Normal	Warning	Critical
Vehicle Location	Valid GPS location	GPS signal weak	Location unavailable
Vehicle Movement	Normal	Unusual movement	Unauthorized movement
Communication	Connected	Weak/intermittent	Communication lost
Route	Assigned route	Route deviation	Major deviation
GPS Signal	Strong	Weak	No GPS signal
Fleet Status	Active	Delayed	Vehicle unavailable

PARAMETERS MONITORED

Parameter	Sensor/Device	Purpose
Vehicle Location	GPS Module	Determine vehicle position
Latitude	GPS Module	Identify north/south position
Longitude	GPS Module	Identify east/west position
Vehicle Movement	GPS Module	Monitor vehicle movement
GPS Signal	GPS Module	Check location availability
Communication Status	GSM/Wi-Fi Module	Monitor data transmission
Vehicle Status	Microcontroller/Cloud	Display vehicle condition
Route Status	Monitoring Platform	Monitor route compliance

CIRCUIT DESIGN



ALGORITHM

- **START**
- Initialize the microcontroller.
- Initialize the GPS module.
- Initialize the communication module.
- Establish communication with the monitoring platform.
- Wait for valid GPS data.
- Read latitude and longitude.
- Read vehicle location information.
- Process the GPS data.
- Check communication status.
- Transmit vehicle location data to the monitoring platform.
- Update the vehicle position on the dashboard.
- Check vehicle route/status.
- If the vehicle deviates from the assigned route, generate a warning.
- Continue monitoring the vehicle.
- Repeat the process continuously.
- **STOP/REPEAT**

CODING

```
#include <ESP8266WiFi.h>
#include <TinyGPS++.h>
#include <SoftwareSerial.h>
#include <DHT.h>

#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// =====
// WIFI
// =====

#define WIFI_SSID    "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// =====
// ADAFRUIT IO
// =====

#define AIO_SERVER    "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "Hair_13"
#define AIO_KEY        "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// =====
// DHT11
// =====

#define DHTPIN 13    // D7
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// =====
// MQ-2
// Digital output only
// =====
```

```

#define MQ2_DO 0      // D3

// Most MQ-2 modules:
// LOW  = gas detected
// HIGH = normal
// =====

// =====
// PULSE SENSOR
// =====

#define PULSE_PIN A0

// =====
// BUZZER
// =====

#define BUZZER_PIN 16 // D0

// =====
// GPS
//
// GPS TX -> ESP8266 D5
// GPS RX -> ESP8266 D6
// =====

#define GPS_RX 14      // D5
#define GPS_TX 12      // D6

TinyGPSPlus gps;

SoftwareSerial gpsSerial(
  GPS_RX,
  GPS_TX
);

// =====
// WIFI CLIENT
// =====

```

```
WiFiClient client;
```

```
// =====
```

```
// MQTT
```

```
// =====
```

```
Adafruit_MQTT_Client mqttt(
```

```
  &client,
```

```
  AIO_SERVER,
```

```
  AIO_SERVERPORT,
```

```
  AIO_USERNAME,
```

```
  AIO_KEY
```

```
);
```

```
// =====
```

```
// ACTIVE ADAFRUIT FEEDS
```

```
// =====
```

```
// Gas
```

```
Adafruit_MQTT_Publish gasFeed =
```

```
  Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/gas"
```

```
  );
```

```
// Heart rate
```

```
Adafruit_MQTT_Publish heartRateFeed =
```

```
  Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/heart-rate"
```

```
  );
```

```
// Heart rate graph
```

```
Adafruit_MQTT_Publish heartRateGraphFeed =
```

```
  Adafruit_MQTT_Publish(
```

```
    &mqtt,
```

```
    AIO_USERNAME "/feeds/heart-rate-graph"
```

```
  );
```

```

// Temperature
Adafruit_MQTT_Publish temperatureFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/temperature"
  );

// Humidity
Adafruit_MQTT_Publish humidityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/humidity"
  );

// Worker status
Adafruit_MQTT_Publish workerStatusFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/worker-status"
  );

// Altitude
Adafruit_MQTT_Publish altitudeFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/altitude"
  );

// GPS Map
Adafruit_MQTT_Publish gpsMapFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/gps-map"
  );

// =====
// SENSOR VARIABLES
// =====

float temperature = 0;

```

```
float humidity = 0;

int gasState = 0;

int heartRate = 0;

float altitude = 0;

double latitude = 0;
double longitude = 0;

int gpsSatellites = 0;

bool gpsFix = false;

// =====
// TIMERS
// =====

unsigned long lastUpload = 0;
unsigned long lastDHTRead = 0;
unsigned long lastGPSPrint = 0;

const unsigned long UPLOAD_INTERVAL = 30000;
const unsigned long DHT_INTERVAL = 2000;

// =====
// HEART RATE VARIABLES
// =====

int pulseThreshold = 550;

bool pulseDetected = false;

unsigned long lastBeatTime = 0;

int beatCount = 0;

unsigned long beatTimes[10];
```

```

int beatIndex = 0;

// =====
// WIFI
// =====

void connectWiFi()
{
  if (WiFi.status() == WL_CONNECTED)
    return;

  Serial.println();
  Serial.println("Connecting to WiFi...");

  WiFi.mode(WIFI_STA);

  WiFi.begin(
    WIFI_SSID,
    WIFI_PASSWORD
  );

  unsigned long start = millis();

  while (
    WiFi.status() != WL_CONNECTED &&
    millis() - start < 15000
  )
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();

  if (WiFi.status() == WL_CONNECTED)
  {
    Serial.println("WIFI CONNECTED");

    Serial.print("IP: ");
    Serial.println(WiFi.localIP());
  }
}

```

```

    }
    else
    {
        Serial.println("WIFI CONNECTION FAILED");
    }
}

// =====
// MQTT
// =====

bool connectMQTT()
{
    if (mqtt.connected())
        return true;

    if (WiFi.status() != WL_CONNECTED)
    {
        connectWiFi();

        if (WiFi.status() != WL_CONNECTED)
            return false;
    }

    Serial.println();
    Serial.println("Connecting to Adafruit IO...");

    int8_t ret;

    for (int i = 0; i < 3; i++)
    {
        ret = mqtt.connect();

        if (ret == 0)
        {
            Serial.println("ADAFRUIT IO CONNECTED");
            return true;
        }

        Serial.print("MQTT ERROR: ");

```

```
Serial.println(
  mqtt.connectErrorString(ret)
);

mqtt.disconnect();

delay(2000);
}

return false;
}

// =====
// READ GPS
// =====

void readGPS()
{
  while (gpsSerial.available())
  {
    char c = gpsSerial.read();

    gps.encode(c);
  }

  if (gps.location.isValid())
  {
    gpsFix = true;

    latitude =
      gps.location.lat();

    longitude =
      gps.location.lng();
  }

  if (gps.altitude.isValid())
  {
    altitude =
      gps.altitude.meters();
  }
}
```

```

    }

    if (gps.satellites.isValid())
    {
        gpsSatellites =
            gps.satellites.value();
    }
}

// =====
// READ DHT11
// =====

void readDHT()
{
    if (
        millis() - lastDHTRead <
        DHT_INTERVAL
    )
    {
        return;
    }

    lastDHTRead = millis();

    float newHumidity =
        dht.readHumidity();

    float newTemperature =
        dht.readTemperature();

    if (
        isnan(newHumidity) ||
        isnan(newTemperature)
    )
    {
        Serial.println("DHT11 READ FAILED");
        return;
    }
}

```

```

    humidity =
        newHumidity;

    temperature =
        newTemperature;
}

// =====
// READ MQ-2
// =====

void readGas()
{
    int mq2Value =
        digitalRead(MQ2_DO);

    if (mq2Value == LOW)
    {
        gasState = 1;
    }
    else
    {
        gasState = 0;
    }
}

// =====
// READ PULSE SENSOR
// =====

void readPulse()
{
    int signal =
        analogRead(PULSE_PIN);

    unsigned long now =
        millis();

    // Detect rising pulse
    if (

```

```
    signal > pulseThreshold &&
    !pulseDetected
)
{
    pulseDetected = true;

    if (lastBeatTime > 0)
    {
        unsigned long interval =
            now - lastBeatTime;

        // Accept realistic heartbeat intervals
        if (
            interval >= 300 &&
            interval <= 2000
        )
        {
            int bpm =
                60000 / interval;

            // Store beat time
            beatTimes[beatIndex] =
                now;

            beatIndex++;

            if (beatIndex >= 10)
                beatIndex = 0;

            beatCount++;

            // Simple averaging
            int validCount = 0;

            unsigned long total =
                0;

            for (int i = 0; i < 10; i++)
            {
                if (
```

```

        beatTimes[i] > 0 &&
        now - beatTimes[i] < 15000
    )
    {
        validCount++;
    }
}

// Use current BPM initially
if (
    bpm >= 40 &&
    bpm <= 180
)
{
    heartRate = bpm;
}
}

lastBeatTime = now;
}

// Wait until signal falls
if (
    signal < pulseThreshold - 50
)
{
    pulseDetected = false;
}
}

// =====
// WORKER STATUS
// =====

String getWorkerStatus()
{
    // Gas alert
    if (gasState == 1)
    {

```

```
    return "ALERT";
}

// Heart rate abnormal
if (
    heartRate > 0 &&
    (
        heartRate < 50 ||
        heartRate > 120
    )
)
{
    return "ALERT";
}

return "SAFE";
}

// =====
// BUZZER
// =====

void updateBuzzer()
{
    String status =
        getWorkerStatus();

    if (status == "ALERT")
    {
        digitalWrite(
            BUZZER_PIN,
            HIGH
        );
    }
    else
    {
        digitalWrite(
            BUZZER_PIN,
            LOW
        );
    }
}
```

```

    }
}

// =====
// PRINT SENSOR DATA
// =====

void printSensorData()
{
    Serial.println();
    Serial.println("=====");
    Serial.println(" WORKER HEALTH MONITOR");
    Serial.println("=====");

    Serial.print("Temperature : ");
    Serial.print(temperature, 1);
    Serial.println(" C");

    Serial.print("Humidity   : ");
    Serial.print(humidity, 1);
    Serial.println(" %");

    Serial.print("Gas       : ");

    if (gasState == 1)
        Serial.println("ALERT");
    else
        Serial.println("NORMAL");

    Serial.print("Heart Rate : ");

    if (heartRate > 0)
    {
        Serial.print(heartRate);
        Serial.println(" BPM");
    }
    else
    {
        Serial.println("Waiting...");
    }
}

```

```
Serial.print("Status   : ");
Serial.println(
  getWorkerStatus()
);

if (gpsFix)
{
  Serial.print("Latitude   : ");
  Serial.println(
    latitude,
    6
  );

  Serial.print("Longitude  : ");
  Serial.println(
    longitude,
    6
  );

  Serial.print("Altitude   : ");
  Serial.print(
    altitude,
    2
  );

  Serial.println(" m");

  Serial.print("Satellites : ");
  Serial.println(
    gpsSatellites
  );
}
else
{
  Serial.println("GPS       : NO FIX");
}

Serial.println("=====");
}
```

```
// =====  
// SEND TO ADAFRUIT IO  
// =====  
  
void sendToAdafruit()  
{  
  if (!connectMQTT())  
  {  
    Serial.println(  
      "MQTT NOT CONNECTED"  
    );  
  
    return;  
  }  
  
  Serial.println();  
  Serial.println(  
    "Uploading sensor data..."  
  );  
  
  // =====  
  // GAS  
  // =====  
  
  if (  
    gasFeed.publish(gasState)  
  )  
  {  
    Serial.print("Gas -> ");  
    Serial.println(gasState);  
  }  
  else  
  {  
    Serial.println("Gas -> FAILED");  
  }  
  
  delay(300);  
  
  // =====
```

```
// HEART RATE
// =====

if (heartRate > 0)
{
  if (
    heartRateFeed.publish(
      heartRate
    )
  )
  {
    Serial.print("Heart Rate -> ");
    Serial.println(heartRate);
  }

  delay(300);

  // Heart rate graph
  if (
    heartRateGraphFeed.publish(
      heartRate
    )
  )
  {
    Serial.print(
      "Heart Rate Graph -> "
    );

    Serial.println(
      heartRate
    );
  }
}

delay(300);

// =====
// TEMPERATURE
// =====
```

```
if (  
    temperatureFeed.publish(  
        temperature  
    )  
)  
{  
    Serial.print("Temperature -> ");  
    Serial.println(  
        temperature,  
        1  
    );  
}
```

```
delay(300);
```

```
// =====  
// HUMIDITY  
// =====
```

```
if (  
    humidityFeed.publish(  
        humidity  
    )  
)  
{  
    Serial.print("Humidity -> ");  
    Serial.println(  
        humidity,  
        1  
    );  
}
```

```
delay(300);
```

```
// =====  
// WORKER STATUS  
// =====
```

```
String status =  
    getWorkerStatus();
```

```

if (
  workerStatusFeed.publish(
    status.c_str()
  )
)
{
  Serial.print("Worker Status -> ");
  Serial.println(status);
}

delay(300);

// =====
// ALTITUDE
// =====

if (gpsFix)
{
  if (
    altitudeFeed.publish(
      altitude
    )
  )
  {
    Serial.print("Altitude -> ");
    Serial.println(
      altitude,
      2
    );
  }
}

delay(300);

// =====
// GPS MAP
// =====

String gpsData =
  "{\"value\":1,\"lat\":\" +

```

```

String(latitude, 6) +
",\"lon\":\" +
String(longitude, 6) +
",\"ele\":\" +
String(altitude, 2) +
"}";

if (
  gpsMapFeed.publish(
    gpsData.c_str()
  )
)
{
  Serial.println(
    "GPS Map -> UPDATED"
  );
}

Serial.println();
Serial.println(
  "UPLOAD COMPLETE"
);
}

// =====
// SETUP
// =====

void setup()
{
  Serial.begin(115200);

  delay(1500);

  Serial.println();
  Serial.println(
    "=====
  );

```

```
Serial.println(  
  " WORKER HEALTH & SAFETY MONITORING"  
);
```

```
Serial.println(  
  "===== "  
);
```

```
// -----  
// PINS  
// -----
```

```
pinMode(  
  MQ2_DO,  
  INPUT  
);
```

```
pinMode(  
  BUZZER_PIN,  
  OUTPUT  
);
```

```
digitalWrite(  
  BUZZER_PIN,  
  LOW  
);
```

```
// -----  
// DHT  
// -----
```

```
dht.begin();
```

```
// -----  
// GPS  
// -----
```

```
gpsSerial.begin(9600);
```

```
Serial.println(  
  "===== "  
);
```

```

    "GPS initialized"
);

Serial.println(
    "GPS TX -> D5"
);

Serial.println(
    "GPS RX -> D6"
);

// -----
// WIFI
// -----

connectWiFi();

Serial.println();
Serial.println(
    "Sensors initialized."
);

Serial.println(
    "Place finger on Pulse Sensor."
);

Serial.println(
    "Waiting for GPS fix..."
);
}

// =====
// LOOP
// =====

void loop()
{
    // =====
    // GPS
    // =====

```

```
readGPS();
```

```
// =====
```

```
// DHT11
```

```
// =====
```

```
readDHT();
```

```
// =====
```

```
// MQ2
```

```
// =====
```

```
readGas();
```

```
// =====
```

```
// PULSE SENSOR
```

```
// =====
```

```
readPulse();
```

```
// =====
```

```
// BUZZER
```

```
// =====
```

```
updateBuzzer();
```

```
// =====
```

```
// PRINT EVERY 2 SECONDS
```

```
// =====
```

```
if (  
  millis() - lastGPSPrint >= 2000  
)  
{  
  lastGPSPrint =  
    millis();  
  
  printSensorData();  
}
```

```

// =====
// ADAFRUIT UPLOAD EVERY 30 SECONDS
// =====

if (
  millis() - lastUpload >=
  UPLOAD_INTERVAL
)
{
  lastUpload =
    millis();

  sendToAdafruit();
}

// =====
// MQTT
// =====

if (mqtt.connected())
{
  mqtt.processPackets(10);
  mqtt.ping();
}

delay(10);
}

```

SYSTEM WORKING

The GPS module installed in the vehicle continuously receives signals from GPS satellites. It calculates the current geographical position of the vehicle in terms of latitude and longitude. The microcontroller receives the GPS information and processes the location data. A communication module such as GSM, Wi-Fi, or another IoT communication interface transmits the information to a remote monitoring platform. The fleet management dashboard receives the data and displays the current vehicle location. The fleet operator can monitor vehicle movement and track multiple vehicles. If the vehicle moves away from its assigned route or if communication/GPS information becomes unavailable, the system can provide an appropriate warning.

COMPONENTS

S.No.	Component	Quantity	Purpose
1	Arduino/ESP32 Microcontroller	1	Main controller
2	GPS Module	1	Vehicle location tracking
3	GSM/Wi-Fi Module	1	Data communication
4	Antenna	As required	GPS/communication signal
5	LCD/OLED Display	1	Display vehicle information
6	Buzzer	1	Warning indication
7	LEDs	As required	Status indication
8	Power Supply	1	Power source
9	Breadboard	1	Circuit assembly
10	Jumper Wires	1 set	Circuit connections
11	Computer/Mobile Dashboard	1	Fleet monitoring

PROTOTYPE IMPLEMENTATION

The prototype can be implemented using an ESP32/Arduino microcontroller, GPS module, communication module, display, LEDs, and buzzer. The GPS module is installed on the vehicle model and connected to the microcontroller. The GPS module receives satellite signals and provides the vehicle's latitude and longitude. The microcontroller processes the GPS information and sends it through the communication module to a cloud-based or local monitoring platform. The monitoring dashboard displays the current vehicle location and fleet status. Multiple vehicle units can be represented in the dashboard to demonstrate fleet management. The prototype can be simulated or demonstrated using suitable IoT and circuit-simulation platforms.



RESULTS & PERFORMANCE ANALYSIS

Condition	GPS/Vehicle Status	System Action
Vehicle stationary	Valid location	Location displayed
Vehicle moving	Valid location	Position continuously updated
GPS signal weak	Reduced accuracy	Warning indication
GPS unavailable	No valid location	GPS error/warning
Vehicle follows route	Normal	No alert
Vehicle deviates from route	Route deviation	Warning generated
Communication lost	Data unavailable	Communication warning

PERFORMANCE

- Real-time vehicle tracking
- Continuous location monitoring
- Latitude and longitude monitoring
- Vehicle movement tracking
- Remote fleet monitoring
- Route monitoring
- Route-deviation detection
- Centralized fleet management
- Vehicle status monitoring
- Improved fleet utilization
- Reduced manual monitoring
- Improved transportation management
- Enhanced vehicle visibility
- Low-cost prototype suitable for educational and simulation applications

